

01-认识 ansible

1 ansible 特点

ansible 特点：

1. 管理的设备类型多，包含云、网络设备、服务器设备（Linux、windows）、存储设备
2. 通过设备自带的原生态方式进行接管，不需要在被管理节点上来安装代理或者客户端软件
3. 无中心架构，不存在什么中心节点，换句话说安装了 ansible 工具的节点都是控制节点
4. ansible 仅仅是一个工具，不是什么服务，所以不需要启动服务来运行 ansible
5. ansible 属于模块化的自动化工具，默认是 python 开发的，支持众多的模块（ansible 支持的模块可以是其他的语言开发的）

Ansible 是一个开源的项目，ansible 的社区也很活跃

属于红帽旗下的开源项目，ansible 对于红帽来说非常的看重，在红帽自家的产品中很多都是使用 ansible 部署的，在红帽自家的培训课程，很多课程都是和 ansible 相关的

比如红帽的分布式存储产品 Ceph：使用 ansible 来自动化部署

比如红帽的容器平台 openshift：使用 ansible 来自动化部署

....

RHCE（rh294 课程代码）课程全部讲的都是 ansible

RHCA：

DO447 老版本

DO374 新版本

讲的都是企业级别的自动化平台 底层是 ansible 实现的

447：ansible automation platform 第一代的自动化平台

374：ansible automation platform2 第二代的自动化平台

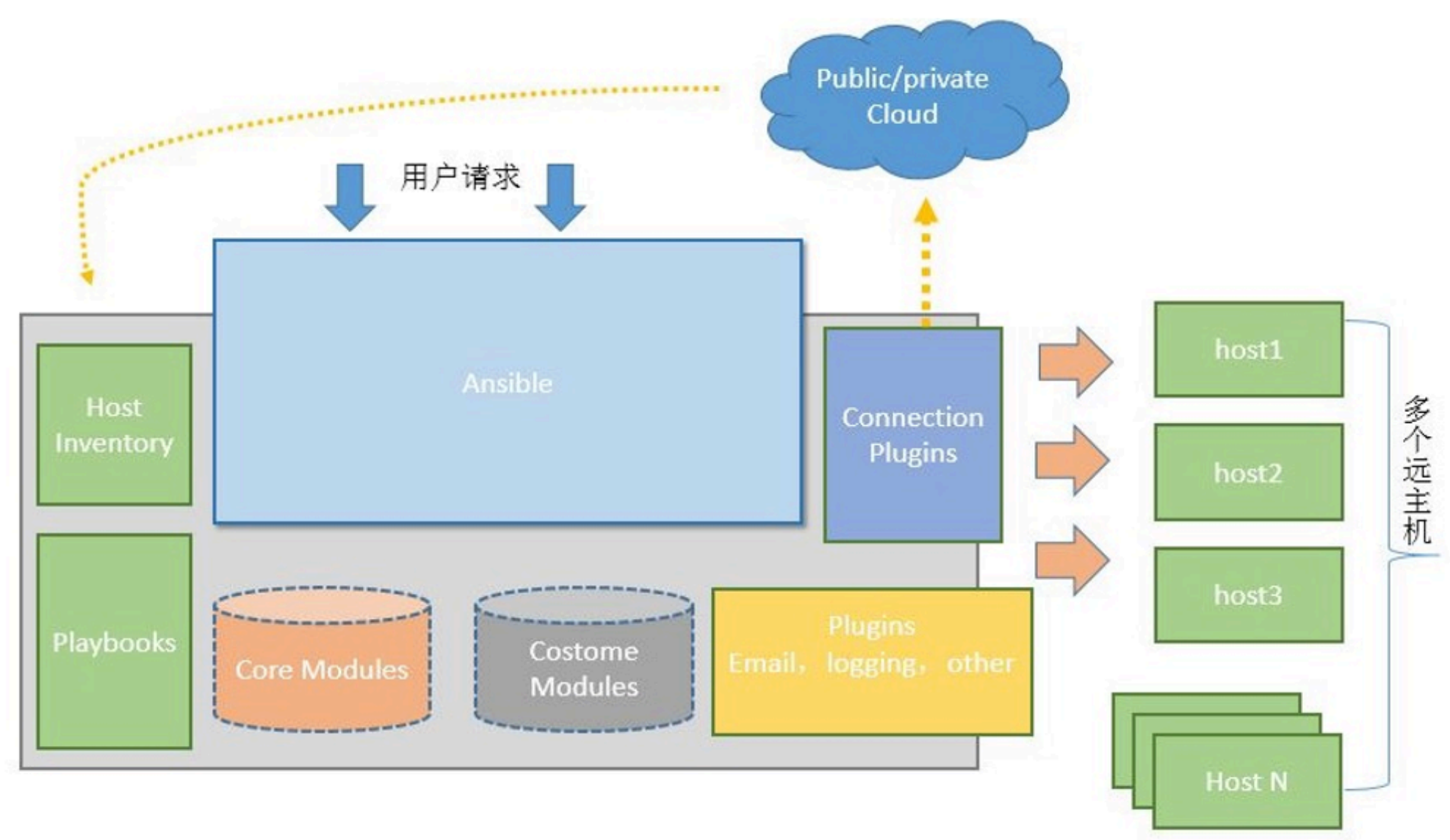
RH358：服务管理（dhcp、dns、nginx、httpd、samba、varish、postfix..），然后将 ansible 怎么实现自动化管理这些服务

DO：DevOps

RH：基础架构

CL：云平台

2 ansible 架构



ansible: 核心工具

inventory: 文件, 主机清单, ansible 只能管理主机清单中的被控节点

modules: 模块。ansible 通过调用模块来管理被控节点

比如调用 user 模块可以管理被控节点的用户、调用 yum 模块可以在被控节点安装软件包、调用 firewalld 模块可以管理被控节点的防护墙...

core-modules: 核心模块, ansible 自带的模块

costome-modules: 扩展模块, 属于用户自己安装的模块, 这些模块要么是第三方企业公司开发的, 要么就是个人开发者开发的

plugins: 插件: 对 ansible 功能的补充; 比如怎么连接到被控节点的, 比如执行任务的时候我想要记录它执行了什么, 比如执行完任务之后我想知道这个任务最终消耗了多少 CPU 内存等资源

连接插件

扩展插件

邮件插件

...

通过 playbook 在所有的被控节点实现自动创建用户和设置密码的任务

YAML 复制

```
[root@aa ansible]# cat useradd-zhangsan.yaml
- name: create user
  hosts: all
  tasks:
    - name: create zhangsan user
      user:
        name: zhangsan
        state: present
    - name: modify user password
      shell: echo 1 | passwd --stdin zhangsan
```

1. 读取 playbook 中的文件, 得到了要在哪些主机上执行哪些任务
2. 读取 inventory 主机清单中的主机
3. 将任务封装为 py 脚本文件, 通过连接插件发送到被控节点

- 4. 被控节点基于脚本文件实现任务执行，执行完成之后删除脚本文件
- 5. 控制节点得到反馈信息

3 ansible 的变化

原来 ansible 就真的只是 ansible，不过从 ansible2.9 版本之后，ansible 不再是一个完整的 ansible 工具了

ansible2.9 版本之后，对 ansible 进行了拆分

拆分出来了俩个东西：

ansible-core：就是 ansible 工具，自带核心模块 大概 70 左右个。很多常用模块都没有，比如 firewalld 模块...

collection：集合，为 ansible 提供三个东西：modules、plugins、roles

因为容器的兴起，现在的 ansible 也已经靠近了容器化，我们在红帽 9 中会学到通过容器方式来执行 ansible 任务，而这种就叫做 ansible-navigator（导航器）

ansible-navigator run useradd-zhangsan.yaml -->运行一个临时容器，在容器中调用 ansible 命令来执行这个 playbook 然后在被控节点执行任务

4 ansible 的安装

现在不谈容器化的 ansible（先不谈导航器）

看看 ansible 和 ansible-core 如何安装

4.1 基于 rpm 包安装

4.1.1 本地 ISO 镜像文件

RHEL9 版本携带的是 ansible-core 版本

Bash 复制

```
[root@adsl-172-10-0-153 ~]# ansible --version
ansible [core 2.14.9]
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/root/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3.9/site-packages/ansible
  ansible collection location = /root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.9.18 (main, Sep  7 2023, 00:00:00) [GCC 11.4.1 20230605 (Red Hat 11.4.1-2)] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
```

4.1.2 通过网络仓库安装 ansible-core

网络仓库上：比如阿里云镜像站中就可以安装。网络仓库安装的 ansible-core 的同时，还会安装很多 collection，它会以依赖的方式同时安装，所以网络安装 ansible-core 的话，携带的模块就很多

三

阿里云

大模型

产品

解决方案

权益

定价

云市场

伙伴

服务

了解阿里云

大模型

🌟

🔗

🔊

文档

备案

控制台

注册

登录

开发者社区

|

首页

探索云世界

问产品

动手实践

官方微博

考认证

TIANCHI大赛

活动广场

下载

开发者社区 > 镜像站 > epel

Epel 镜像

简介

EPEL(Extra Packages for Enterprise Linux), 是由Fedora Special Interest Group维护的Enterprise Linux (RHEL、CentOS) 中经常用到的包。

下载地址：<https://mirrors.aliyun.com/epel/>

相关仓库

- epel过期源 (epel-archive)：<https://developer.aliyun.com/mirror/epel-archive>

配置方法

1. 备份(如有配置其他epel源)

```
mv /etc/yum.repos.d/epel.repo /etc/yum.repos.d/epel.repo.backup
mv /etc/yum.repos.d/epel-testing.repo /etc/yum.repos.d/epel-testing.repo.backup
```

2. 下载新repo 到/etc/yum.repos.d/
epel(RHEL 8)

镜像站

OPsx

阿里巴巴镜像站技术社区

+ 订阅

域名使用规则

- 公网访问地址:
<https://mirrors.aliyun.com/>
- ECS VPC网络访问地址:
<http://mirrors.cloud.aliyuncs.com/>
- ECS 经典网络访问地址:
<http://mirrors.aliyuncs.com/>

对于镜像有任何问题，欢迎给我们提问题！

立即提问

热门解决方案

- 快速部署 Dify，高效搭建 AI 应用
- 通义：云服务器ECS免费得
- 快速搭建微信、支付宝小程序
- 低成本高效构建企业门户网站

https://mirrors.aliyun.com/epel-archive/9.3/Everything/x86_64/

YAML

复制

```
[epel]
name=epel
baseurl=https://mirrors.aliyun.com/epel-archive/9.3/Everything/x86_64/
gpgcheck=0
enabled=1
```

这里实际上提供的是 ansible 的集合，ansible-core 还是来自于本地

你也可以选择配置红帽的网络仓库，这样的话无论是 ansible-core 还是集合都是从网络仓库安装的

4.2 通过 pip 工具安装

这种安装方式最简单！只需要一条命令，什么都安装好了

pip 是 python 的包管理工具，因为 ansible 是 python 开发的，所以 pip 可以安装

Bash

复制

```
[root@adsl-172-10-0-153 ~]# pip install ansible==2.9.0 -i https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
```

pip 工具本质上就是通过 pip 工具然后下载了 ansible 的源码包，实现了自动安装

Bash

复制

```
[root@adsl-172-10-0-153 ~]# /usr/local/bin/ansible --version
ansible 2.9.0
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/local/lib/python3.9/site-packages/ansible
  executable location = /usr/local/bin/ansible
  python version = 3.9.18 (main, Sep 7 2023, 00:00:00) [GCC 11.4.1 20230605 (Red Hat 11.4.1-2)]
```

pip 安装的 ansible 没有配置文件模板

在网络上去找 ansible.cfg 配置文件模板

4.3 通过 ansible 源码包实现安装

ansible 的源码包在俩个地方可以下载到：

<https://releases.ansible.com/>: 下载的 ansible 源码包只维护到 2.9 版本

<https://github.com/ansible/ansible>: 下载任意版本的 ansible 源码包

我告诉大家，这里的源码包安装依旧需要从网上下载依赖软件

Bash 复制

```
wget https://releases.ansible.com/ansible/ansible-2.9.0.tar.gz
```

build 构建、install 安装

Python 复制

```
python setup.py build
python setup.py install
```

源码包安装之后，默认没有配置文件。提供了配置文件模板在目录下

Bash 复制

```
[root@adsl-172-10-0-153 ansible-2.9.0]# ls
bin          COPYING    hacking    MANIFEST.in  requirements.txt  test
build        dist       lib        packaging    setup.py
changelogs   docs       licenses  PKG-INFO     shippable.yml
contrib      examples   Makefile   README.rst   SYMLINK_CACHE.json
[root@adsl-172-10-0-153 ansible-2.9.0]# ls examples/
ansible.cfg  hosts  scripts
[root@adsl-172-10-0-153 ansible-2.9.0]# vim examples/ansible.cfg
```

4.4 ansible.cfg 配置文件模板（2.9.0 版本）

Markdown 复制

```
# config file for ansible -- https://ansible.com/
# =====

# nearly all parameters can be overridden in ansible-playbook
# or with command line flags.  ansible will read ANSIBLE_CONFIG,
# ansible.cfg in the current working directory, .ansible.cfg in
# the home directory or /etc/ansible/ansible.cfg, whichever it
# finds first

[defaults]

# some basic default values...

#inventory      = /etc/ansible/hosts
#library        = /usr/share/my_modules/
#module_utils   = /usr/share/my_module_utils/
#remote_tmp     = ~/.ansible/tmp
#local_tmp      = ~/.ansible/tmp
#plugin_filters_cfg = /etc/ansible/plugin_filters.yml
#forks          = 5
#poll_interval  = 15
#sudo_user      = root
#ask_sudo_pass  = True
#ask_pass       = True
#transport      = smart
#remote_port    = 22
#module_lang    = C
#module_set_locale = False

# plays will gather facts by default, which contain information about
# the remote system.
#
# smart - gather by default, but don't regather if already gathered
# implicit - gather by default, turn off with gather_facts: False
# explicit - do not gather by default, must say gather_facts: True
#gathering = implicit
```

```
# This only affects the gathering done by a play's gather_facts directive,
# by default gathering retrieves all facts subsets
# all - gather all subsets
# network - gather min and network facts
# hardware - gather hardware facts (longest facts to retrieve)
# virtual - gather min and virtual facts
# facter - import facts from facter
# ohai - import facts from ohai
# You can combine them using comma (ex: network,virtual)
# You can negate them using ! (ex: !hardware,!facter,!ohai)
# A minimal set of facts is always gathered.
#gather_subset = all

# some hardware related facts are collected
# with a maximum timeout of 10 seconds. This
# option lets you increase or decrease that
# timeout to something more suitable for the
# environment.
# gather_timeout = 10

# Ansible facts are available inside the ansible_facts.* dictionary
# namespace. This setting maintains the behaviour which was the default prior
# to 2.5, duplicating these variables into the main namespace, each with a
# prefix of 'ansible_'.
# This variable is set to True by default for backwards compatibility. It
# will be changed to a default of 'False' in a future release.
# ansible_facts.
# inject_facts_as_vars = True

# additional paths to search for roles in, colon separated
#roles_path    = /etc/ansible/roles

# uncomment this to disable SSH key host checking
#host_key_checking = False

# change the default callback, you can only have one 'stdout' type  enabled at a time.
#stdout_callback = skippy

## Ansible ships with some plugins that require whitelisting,
## this is done to avoid running all of a type by default.
## These setting lists those that you want enabled for your system.
## Custom plugins should not need this unless plugin author specifies it.

# enable callback plugins, they can output to stdout but cannot be 'stdout' type.
#callback_whitelist = timer, mail

# Determine whether includes in tasks and handlers are "static" by
# default. As of 2.0, includes are dynamic by default. Setting these
# values to True will make includes behave more like they did in the
# 1.x versions.
#task_includes_static = False
#handler_includes_static = False

# Controls if a missing handler for a notification event is an error or a warning
#error_on_missing_handler = True

# change this for alternative sudo implementations
#sudo_exe = sudo

# What flags to pass to sudo
# WARNING: leaving out the defaults might create unexpected behaviours
#sudo_flags = -H -S -n

# SSH timeout
#timeout = 10

# default user to use for playbooks if user is not specified
# (/usr/bin/ansible will use current user as default)
#remote_user = root
```

```
# logging is off by default unless this path is defined
# if so defined, consider logrotate
#log_path = /var/log/ansible.log

# default module name for /usr/bin/ansible
#module_name = command

# use this shell for commands executed under sudo
# you may need to change this to bin/bash in rare instances
# if sudo is constrained
#executable = /bin/sh

# if inventory variables overlap, does the higher precedence one win
# or are hash values merged together? The default is 'replace' but
# this can also be set to 'merge'.
#hash_behaviour = replace

# by default, variables from roles will be visible in the global variable
# scope. To prevent this, the following option can be enabled, and only
# tasks and handlers within the role will see the variables there
#private_role_vars = yes

# list any Jinja2 extensions to enable here:
#jinja2_extensions = jinja2.ext.do,jinja2.ext.i18n

# if set, always use this private key file for authentication, same as
# if passing --private-key to ansible or ansible-playbook
#private_key_file = /path/to/file

# If set, configures the path to the Vault password file as an alternative to
# specifying --vault-password-file on the command line.
#vault_password_file = /path/to/vault_password_file

# format of string {{ ansible_managed }} available within Jinja2
# templates indicates to users editing templates files will be replaced.
# replacing {file}, {host} and {uid} and strftime codes with proper values.
#ansible_managed = Ansible managed: {file} modified on %Y-%m-%d %H:%M:%S by {uid} on {host}
# {file}, {host}, {uid}, and the timestamp can all interfere with idempotence
# in some situations so the default is a static string:
#ansible_managed = Ansible managed

# by default, ansible-playbook will display "Skipping [host]" if it determines a task
# should not be run on a host. Set this to "False" if you don't want to see these "Skipping"
# messages. NOTE: the task header will still be shown regardless of whether or not the
# task is skipped.
#display_skipped_hosts = True

# by default, if a task in a playbook does not include a name: field then
# ansible-playbook will construct a header that includes the task's action but
# not the task's args. This is a security feature because ansible cannot know
# if the *module* considers an argument to be no_log at the time that the
# header is printed. If your environment doesn't have a problem securing
# stdout from ansible-playbook (or you have manually specified no_log in your
# playbook on all of the tasks where you have secret information) then you can
# safely set this to True to get more informative messages.
#display_args_to_stdout = False

# by default (as of 1.3), Ansible will raise errors when attempting to dereference
# Jinja2 variables that are not set in templates or action lines. Uncomment this line
# to revert the behavior to pre-1.3.
#error_on_undefined_vars = False

# by default (as of 1.6), Ansible may display warnings based on the configuration of the
# system running ansible itself. This may include warnings about 3rd party packages or
# other conditions that should be resolved if possible.
# to disable these warnings, set the following value to False:
#system_warnings = True

# by default (as of 1.4), Ansible may display deprecation warnings for language
```

```
# features that should no longer be used and will be removed in future versions.
# to disable these warnings, set the following value to False:
#deprecation_warnings = True


# (as of 1.8), Ansible can optionally warn when usage of the shell and
# command module appear to be simplified by using a default Ansible module
# instead.  These warnings can be silenced by adjusting the following
# setting or adding warn=yes or warn=no to the end of the command line
# parameter string.  This will for example suggest using the git module
# instead of shelling out to the git command.
# command_warnings = False


# set plugin path directories here, separate with colons
#action_plugins      = /usr/share/ansible/plugins/action
#become_plugins      = /usr/share/ansible/plugins/become
#cache_plugins       = /usr/share/ansible/plugins/cache
#callback_plugins    = /usr/share/ansible/plugins/callback
#connection_plugins  = /usr/share/ansible/plugins/connection
#lookup_plugins      = /usr/share/ansible/plugins/lookup
#inventory_plugins   = /usr/share/ansible/plugins/inventory
#vars_plugins        = /usr/share/ansible/plugins/vars
#filter_plugins      = /usr/share/ansible/plugins/filter
#test_plugins        = /usr/share/ansible/plugins/test
#terminal_plugins    = /usr/share/ansible/plugins/terminal
#strategy_plugins    = /usr/share/ansible/plugins/strategy


# by default, ansible will use the 'linear' strategy but you may want to try
# another one
#strategy = free


# by default callbacks are not loaded for /bin/ansible, enable this if you
# want, for example, a notification or logging callback to also apply to
# /bin/ansible runs
#bin_ansible_callbacks = False


# don't like cows?  that's unfortunate.
# set to 1 if you don't want cowsay support or export ANSIBLE_NOCOWS=1
#nocows = 1


# set which cowsay stencil you'd like to use by default. When set to 'random',
# a random stencil will be selected for each task. The selection will be filtered
# against the `cow_whitelist` option below.
#cow_selection = default
#cow_selection = random


# when using the 'random' option for cowsay, stencils will be restricted to this list.
# it should be formatted as a comma-separated list with no spaces between names.
# NOTE: line continuations here are for formatting purposes only, as the INI parser
#       in python does not support them.
#cow_whitelist=bud-frogs,bunny,cheese,daemon,default,dragon,elephant-in-snake,elephant,eyes,\
#              hellokitty,kitty,luke-koala,meow,milk,moofasa,moose,ren,sheep,small,stegosaurus,\
#              stimpy,supermilker,three-eyes,turkey,turtle,tux,udder,vader-koala,vader,www


# don't like colors either?
# set to 1 if you don't want colors, or export ANSIBLE_NOCOLOR=1
#nocolor = 1


# if set to a persistent type (not 'memory', for example 'redis') fact values
# from previous runs in Ansible will be stored.  This may be useful when
# wanting to use, for example, IP information from one group of servers
# without having to talk to them in the same playbook run to get their
# current IP information.
#fact_caching = memory


#This option tells Ansible where to cache facts. The value is plugin dependent.
#For the jsonfile plugin, it should be a path to a local directory.
#For the redis plugin, the value is a host:port:database triplet: fact_caching_connection = localhost:6379:0
```

```
#fact_caching_connection=/tmp

# retry files
# When a playbook fails a .retry file can be created that will be placed in ~/
# You can enable this feature by setting retry_files_enabled to True
# and you can change the location of the files by setting retry_files_save_path

#retry_files_enabled = False
#retry_files_save_path = ~/.ansible-retry

# squash actions
# Ansible can optimise actions that call modules with list parameters
# when looping. Instead of calling the module once per with_ item, the
# module is called once with all items at once. Currently this only works
# under limited circumstances, and only with parameters named 'name'.
#squash_actions = apk,apt,dnf,homebrew,pacman,pkgng,yum,zypper

# prevents logging of task data, off by default
#no_log = False

# prevents logging of tasks, but only on the targets, data is still logged on the master/controller
#no_target_syslog = False

# controls whether Ansible will raise an error or warning if a task has no
# choice but to create world readable temporary files to execute a module on
# the remote machine. This option is False by default for security. Users may
# turn this on to have behaviour more like Ansible prior to 2.1.x. See
# https://docs.ansible.com/ansible/become.html#becoming-an-unprivileged-user
# for more secure ways to fix this than enabling this option.
#allow_world_readable_tmpfiles = False

# controls the compression level of variables sent to
# worker processes. At the default of 0, no compression
# is used. This value must be an integer from 0 to 9.
#var_compression_level = 9

# controls what compression method is used for new-style ansible modules when
# they are sent to the remote system. The compression types depend on having
# support compiled into both the controller's python and the client's python.
# The names should match with the python Zipfile compression types:
# * ZIP_STORED (no compression. available everywhere)
# * ZIP_DEFLATED (uses zlib, the default)
# These values may be set per host via the ansible_module_compression inventory
# variable
#module_compression = 'ZIP_DEFLATED'

# This controls the cutoff point (in bytes) on --diff for files
# set to 0 for unlimited (RAM may suffer!).
#max_diff_size = 1048576

# This controls how ansible handles multiple --tags and --skip-tags arguments
# on the CLI. If this is True then multiple arguments are merged together. If
# it is False, then the last specified argument is used and the others are ignored.
# This option will be removed in 2.8.
#merge_multiple_cli_flags = True

# Controls showing custom stats at the end, off by default
#show_custom_stats = True

# Controls which files to ignore when using a directory as inventory with
# possibly multiple sources (both static and dynamic)
#inventory_ignore_extensions = ~, .orig, .bak, .ini, .cfg, .retry, .pyc, .pyo

# This family of modules use an alternative execution path optimized for network appliances
# only update this setting if you know how this works, otherwise it can break module execution
#network_group_modules=eos, nxos, ios, iosxr, junos, vyos
```

```
# When enabled, this option allows lookups (via variables like {{lookup('foo')}} or when used as
# a loop with `with_foo`) to return data that is not marked "unsafe". This means the data may contain
# jinja2 templating language which will be run through the templating engine.
# ENABLING THIS COULD BE A SECURITY RISK
#allow_unsafe_lookups = False

# set default errors for all plays
#any_errors_fatal = False

[inventory]
# enable inventory plugins, default: 'host_list', 'script', 'auto', 'yaml', 'ini', 'toml'
#enable_plugins = host_list, virtualbox, yaml, constructed

# ignore these extensions when parsing a directory as inventory source
#ignore_extensions = .pyc, .pyo, .swp, .bak, ~, .rpm, .md, .txt, ~, .orig, .ini, .cfg, .retry

# ignore files matching these patterns when parsing a directory as inventory source
#ignore_patterns=

# If 'true' unparsed inventory sources become fatal errors, they are warnings otherwise.
#unparsed_is_failed=False

[privilege_escalation]
#become=True
#become_method=sudo
#become_user=root
#become_ask_pass=False

[paramiko_connection]

# uncomment this line to cause the paramiko connection plugin to not record new host
# keys encountered. Increases performance on new host additions. Setting works independently of the
# host key checking setting above.
#record_host_keys=False

# by default, Ansible requests a pseudo-terminal for commands executed under sudo. Uncomment this
# line to disable this behaviour.
#pty=False

# paramiko will default to looking for SSH keys initially when trying to
# authenticate to remote devices. This is a problem for some network devices
# that close the connection after a key failure. Uncomment this line to
# disable the Paramiko look for keys function
#look_for_keys = False

# When using persistent connections with Paramiko, the connection runs in a
# background process. If the host doesn't already have a valid SSH key, by
# default Ansible will prompt to add the host key. This will cause connections
# running in background processes to fail. Uncomment this line to have
# Paramiko automatically add host keys.
#host_key_auto_add = True

[ssh_connection]

# ssh arguments to use
# Leaving off ControlPersist will result in poor performance, so use
# paramiko on older platforms rather than removing it, -C controls compression use
#ssh_args = -C -o ControlMaster=auto -o ControlPersist=60s

# The base directory for the ControlPath sockets.
# This is the "%(directory)s" in the control_path option
#
# Example:
# control_path_dir = /tmp/.ansible/cp
#control_path_dir = ~/.ansible/cp

# The path to use for the ControlPath sockets. This defaults to a hashed string of the hostname,
# port and username (empty string in the config). The hash mitigates a common problem users
# found with long hostnames and the conventional %(directory)s/ansible-ssh-%%h-%%p-%%r format.
# In those cases, a "too long for Unix domain socket" ssh error would occur.
```

```
#
# Example:
# control_path = %(directory)s/%%h-%%r
#control_path =

# Enabling pipelining reduces the number of SSH operations required to
# execute a module on the remote server. This can result in a significant
# performance improvement when enabled, however when using "sudo:" you must
# first disable 'requiretty' in /etc/sudoers
#
# By default, this option is disabled to preserve compatibility with
# sudoers configurations that have requiretty (the default on many distros).
#
#pipelining = False

# Control the mechanism for transferring files (old)
# * smart = try sftp and then try scp [default]
# * True = use scp only
# * False = use sftp only
#scp_if_ssh = smart

# Control the mechanism for transferring files (new)
# If set, this will override the scp_if_ssh option
# * sftp = use sftp to transfer files
# * scp = use scp to transfer files
# * piped = use 'dd' over SSH to transfer files
# * smart = try sftp, scp, and piped, in that order [default]
#transfer_method = smart

# if False, sftp will not use batch mode to transfer files. This may cause some
# types of file transfer failures impossible to catch however, and should
# only be disabled if your sftp version has problems with batch mode
#sftp_batch_mode = False

# The -tt argument is passed to ssh when pipelining is not enabled because sudo
# requires a tty by default.
#usetty = True

# Number of times to retry an SSH connection to a host, in case of UNREACHABLE.
# For each retry attempt, there is an exponential backoff,
# so after the first attempt there is 1s wait, then 2s, 4s etc. up to 30s (max).
#retries = 3

[persistent_connection]

# Configures the persistent connection timeout value in seconds. This value is
# how long the persistent connection will remain idle before it is destroyed.
# If the connection doesn't receive a request before the timeout value
# expires, the connection is shutdown. The default value is 30 seconds.
#connect_timeout = 30

# The command timeout value defines the amount of time to wait for a command
# or RPC call before timing out. The value for the command timeout must
# be less than the value of the persistent connection idle timeout (connect_timeout)
# The default value is 30 second.
#command_timeout = 30

[accelerate]
#accelerate_port = 5099
#accelerate_timeout = 30
#accelerate_connect_timeout = 5.0

# The daemon timeout is measured in minutes. This time is measured
# from the last activity to the accelerate daemon.
#accelerate_daemon_timeout = 30

# If set to yes, accelerate_multi_key will allow multiple
# private keys to be uploaded to it, though each user must
# have access to the system via SSH to add a new key. The default
# is "no".
```

```
#accelerate_multi_key = yes

[selinux]
# file systems that require special treatment when dealing with security context
# the default behaviour that copies the existing context or uses the user default
# needs to be changed to use the file system dependent context.
#special_context_filesystems=nfs,vboxsf,fuse,ramfs,9p,vfat

# Set this to yes to allow libvirt_lxc connections to work without SELinux.
#libvirt_lxc_noseclabel = yes

[colors]
#highlight = white
#verbose = blue
#warn = bright purple
#error = red
#debug = dark gray
#deprecate = purple
#skip = cyan
#unreachable = red
#ok = green
#changed = yellow
#diff_add = green
#diff_remove = red
#diff_lines = cyan

[diff]
# Always print diff when running ( same as always running with -D/--diff )
# -
```